

Smart MCQ Solver Challenge

Deep Learning and Generative AI Project Report

Aniket Pandey · 23f2003176

August 2026

1. Abstract / Executive Summary

The Smart MCQ Solver Challenge contains a question and five choices, A–E. For every test row, I must submit the three most likely answer letters in order. Mean Average Precision at 3 (MAP@3) gives a better score when the correct answer is nearer the front of this list.

I tried three models: a bidirectional Gated Recurrent Unit (BiGRU) trained from scratch, a pretrained DeBERTa model, and a question-retrieval model using term frequency–inverse document frequency (TF-IDF). I logged each model in Weights & Biases (W&B). On the common validation split, their MAP@3 scores were 0.5590, 0.5131 and 0.5311. A combination that mostly used retrieval produced the Kaggle submission, which scored **0.75519** at rank **293**, above the **0.73** cutoff.

2. Introduction

Problem statement and objective

The training data contains general questions, five answer choices and the correct answer [8]. The test answers are hidden. A valid prediction has three different letters in order, for example A C B.

My goal was to cross the cutoff and compare three different models: one built from scratch, one pretrained model and one extra model. I also wanted a fair validation split, saved model outputs and separate W&B runs so that the results could be checked again.

3. Dataset and Preprocessing

Dataset description and EDA

Split	Rows	Available fields	Labels
Train	2,000	id, prompt, A–E, answer	A 369, B 490, C 459, D 358, E 324
Test	500	id, prompt, A–E	Hidden

The main EDA findings were:

A question family contains different wrapper versions that become the same question after cleaning. I call its cleaned form the canonical prompt.

- There were no missing values.
- Raw prompts averaged 18.15 words (median 17, maximum 51); cleaned prompts averaged 11.72 words (median 11).
- The 2,000 training rows reduced to 252 question families, and 1,970 rows belonged to repeated families.
- All 228 test families also appeared in training.
- Some answer letters occurred more often than others, so I reported Macro F1 along with accuracy.

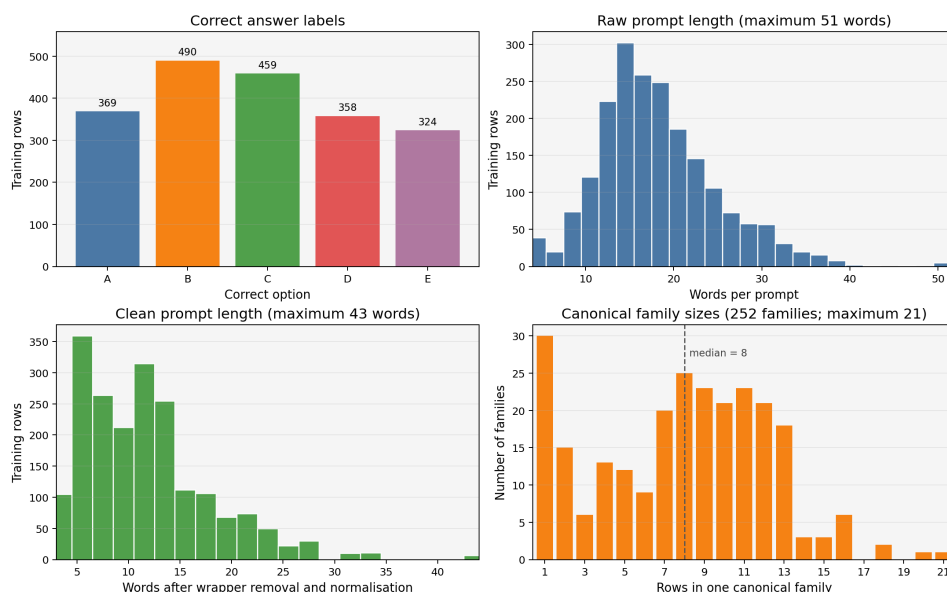


Figure 1. Label balance, prompt lengths and repeated question-family sizes from the executed notebook.

Cleaning and validation

- **Text cleaning:** I used Unicode Normalization Form Compatibility Composition (NFKC) so equivalent characters were written in the same form. I then removed six common prefixes and four suffixes because they were instruction wrappers, not part of the question.
- **Two prompt columns:** `content_prompt` kept case and punctuation for the neural models. `clean_prompt` used lower case, changed punctuation and underscores to spaces, and removed extra spaces. It was used to find question families and similar questions.
- **Validation split:** I used three-fold `StratifiedGroupKFold` [6] with seed 42. It keeps every question family in only one side of a split and also tries to keep the answer-letter balance similar. Fold 0 had 1,277 training rows from 167 families and 723 validation rows from 85 families. No family was present on both sides.
- **No added questions:** I did not generate extra text because the data already contained many natural wrapper versions.

4. Tokenization Strategy

A tokenizer changes text into numbered pieces that a model can read. The scratch BiGRU used byte-pair encoding (BPE) trained only on the training split. BPE repeatedly joins byte pieces that often appear together [4]. This can handle uncommon words without storing every complete word.

Model	Token setting	What entered the model
Scratch BiGRU	BPE vocabulary 5,237; maximum 192 tokens	[BOS] question [SEP] choice [EOS]
DeBERTa	Its own pretrained SentencePiece tokenizer [5]; maximum 128 tokens	Five question-and-choice pairs
Retrieval	No neural tokenizer; groups of 4–6 characters	TF-IDF values

[BOS] marked the start, [SEP] separated the question from an option, and [EOS] marked the end. [PAD] filled empty positions in a batch, while [UNK] represented an unknown piece. A mask told the model which positions contained real text instead of padding.

One BiGRU batch had shape [32,5,142]: 32 questions, five choices for each question and 142 token positions after padding. DeBERTa used the tokenizer supplied with its pretrained files; using a different one would make its token numbers mean the wrong text pieces.

5. Modeling and Experimentation

I built three models. The BiGRU was trained from scratch and is also the recurrent model. Retrieval was the third model.

5.1 Model 1: Fine-tuned DeBERTa-v3-base

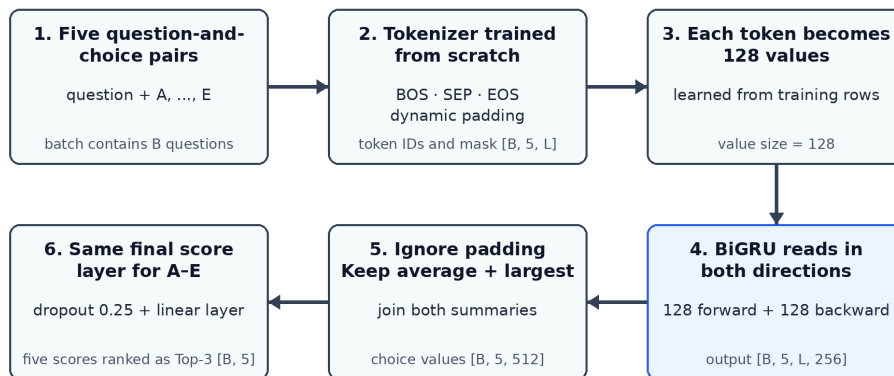
DeBERTa is a language model that was already trained on a large text collection. The base version has 12 layers, 768 values for each token and 12 attention heads [1, 2]. Attention lets a token use useful information from other tokens. DeBERTa also keeps word meaning and word position separate while making this comparison. For each row, I made five question-and-choice pairs. The final layer returned one score for A, B, C, D and E.

Fine-tuning item	Setting
Parts trained	Last four DeBERTa layers and the final answer layers
Parameters	28,942,849 trainable out of 184,422,913
Optimizer	AdamW; learning rate 2×10^{-5} ; weight decay 0.01
Learning-rate change	Increased during the first 10% of steps, then reduced in a straight line
Training	3 epochs; gradient limit 1; batch 1 per GPU; update after 8 small batches
Effective batch	$1 \times 2 \times 8 = 16$ questions on two Kaggle GPUs
Number format	FP16 during training; weights were not changed to 8-bit or 4-bit

Keeping the input part and the first eight layers fixed reduced memory use. Only the last four layers and answer layers learned from this dataset.

5.2 Model 2: Custom deep learning and RNN-based BiGRU

This model started with random weights and used no pretrained model. Its embedding table changed each token number into 128 learned values. A bidirectional GRU then read every sequence in both directions: from left to right and from right to left. Each direction used 128 hidden values. After ignoring padding, I kept both the average and largest GRU output. Together they formed 512 values for each answer choice. During training, dropout randomly hid 25% of these values to reduce memorising. The model had 868,993 trainable parameters.



The same scorer is applied to A-E; its five scores are ranked for the final Top-3.

Figure 2. Architecture of the model trained completely from scratch.

A GRU has gates that decide how much older information to keep or forget [3]. Reading in both directions helps it use words from either side of the sequence.

Training item	Setting
Error measure and optimizer	Cross-entropy; AdamW
Learning rate / weight decay	0.001 / 0.01
Batch / gradient clipping	32 questions / 1.0
Stopping	At most 25 epochs; stop after 5 epochs without improvement
Saved model	The epoch with the best validation MAP@3

One epoch is one full pass over the training rows. After each epoch, I checked validation MAP@3 and saved the best model.

5.3 Model 3: Canonical lookup and character TF-IDF retrieval

This model runs on the CPU and has no training loop. It first stored one cleaned question from each family in the training split. For a new question, it checked for an exact cleaned match. If there was no exact match, character TF-IDF compared short character groups and found similar stored questions. Cosine similarity measured how close two TF-IDF lists were:

$$\text{tf}(t, d) = 1 + \ln c_{t,d}, \quad \text{idf}(t) = \ln\left(\frac{1+n}{1+\text{df}(t)}\right) + 1$$

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

Here t is a character group, d is one question, and $c_{t,d}$ is the number of times that group appears in the question. The value n is the number of stored questions, while $\text{df}(t)$ counts how many stored questions contain the group. TF gives some extra weight to repeated groups. IDF gives more weight to a group that appears in fewer questions. Cosine similarity is near 1 for similar TF-IDF lists and near 0 for very different ones.

I tested character ranges 4–6 and 5–7 with 30 or 50 nearby questions. The best setting used 4–6 characters and 30 neighbours. It gave 0.3900 accuracy, 0.3736 Macro F1 and 0.5311 MAP@3.

- Similar stored questions voted for A–E. The overall answer-letter counts from the training split were used when the match was weak.
- For an exact match, I gave 90% of the score to the stored answer and 10% to the normal fallback scores.
- This model compares the question text, not the meaning of the options. It can fail if the same question is shown with its options in a different order.

6. Performance and Comparative Analysis

Evaluation metrics

MAP@3 means Mean Average Precision at 3. Let r_i be the position of the correct answer for question i . Since every question has one correct answer,

$$\text{MAP@3} = \frac{1}{N} \sum_{i=1}^N s_i$$

$$s_i = 1/r_i \text{ if } r_i \leq 3; \quad s_i = 0 \text{ if } r_i > 3$$

This means the score for one row is 1 when the answer is first, 1/2 when it is second, 1/3 when it is third, and 0 when it is outside the Top 3. For validation ID 1, the true answer was B. BiGRU ranked **E B D**, so this row gave 1/2=0.5. DeBERTa ranked **A D E**, so it gave 0.

Accuracy is the share of rows whose first answer is correct. Macro F1 checks each answer letter separately and then takes the average, so common letters do not control the whole result. For class c , TP_c means rows correctly predicted as c , FP_c means rows wrongly predicted as c , and FN_c means rows of class c that were missed:

$$P_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \quad R_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \quad F1_c = \frac{2 P_c R_c}{P_c + R_c}$$

Macro F1 is the average of the five F1 values, one for A, B, C, D and E.

Common validation results

Model	Best epoch	Accuracy	Macro F1	MAP@3	W&B ID
Scratch BiGRU	1	0.3762	0.3648	0.5590	ztxp2h2f
DeBERTa-v3-base	3	0.3610	0.3596	0.5131	jnozghf9
Canonical + TF-IDF	—	0.3900	0.3736	0.5311	3r7lmej9
Weighted ensemble	—	0.3582	0.3437	0.5226	—

- **BiGRU:** It began overfitting after epoch 1, so I restored that epoch.
- **DeBERTa:** Its validation results improved over all three epochs. Training took about 473.5 seconds.
- **Retrieval:** It had the best accuracy and Macro F1, but it has no epoch-loss curve.

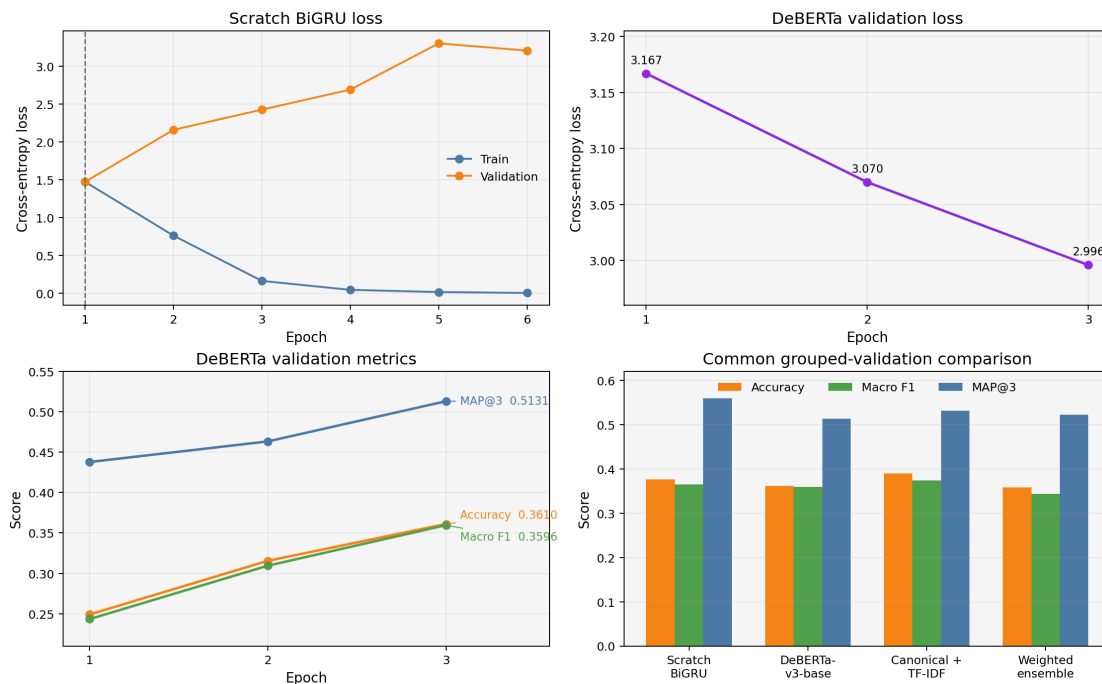


Figure 3. Training and validation results logged in W&B. The last chart compares all four systems on the same validation split.

The final combination used 10% BiGRU, 10% DeBERTa and 80% retrieval. It did **not** beat BiGRU on the validation split. However, validation had no exact family matches, while all 500 test rows matched a family from the full training set. For this reason, I used more retrieval weight for the Kaggle submission. This choice was made for this competition and may not work as well on completely new questions.

Before submission, I checked the ID order, allowed answer letters and three different predictions per row. The public leaderboard MAP@3 was **0.75519** at rank **293**. This was **0.02519** above the 0.73 cutoff. The final private rank may change because it uses a different hidden part of the test set.

7. Conclusion and Future Work

Key learnings

- A larger pretrained model is not always best for a small dataset with many repeated question families.
- Keeping related questions together made validation harder but fairer.
- The retrieval model was useful because the test questions were closely related to training questions.
- Fixed seeds, saved model files, saved predictions and W&B runs made the results easier to check again.

Challenges and solutions

- I stopped similar question families from appearing in both training and validation.
- BiGRU started memorising the training rows, so I stopped early and loaded the best saved epoch.
- DeBERTa did not fit easily in GPU memory, so I kept its early layers fixed, used small batches and combined several small batches before each update.

Future work: Try to improve the Kaggle score by loading larger pretrained models and doing more fine-tuning.

8. References

- **[1]** He, P., Gao, J. and Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.
- **[2]** Microsoft. microsoft/deberta-v3-base official model card.
- **[3]** Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.
- **[4]** Sennrich, R., Haddow, B. and Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units.
- **[5]** Kudo, T. and Richardson, J. (2018). SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer.
- **[6]** scikit-learn documentation: StratifiedGroupKFold and TfidfVectorizer.
- **[7]** Software used: PyTorch, Hugging Face Transformers, Weights & Biases, pandas, NumPy and Matplotlib.
- **[8]** Kaggle. Smart MCQ Solver Challenge.